

Bloque 11 — Mini-especificaciones funcionales y técnicas

Factoring: X001 a X007

1. Propósito del bloque

Este bloque implementa las funciones iniciales necesarias para administrar operaciones de factoring vinculadas con facturas de venta.

Incluye:

- Catálogo de empresas de factoring.
- Contactos y convenios comerciales.
- Cesión de facturas.
- Condiciones particulares de cada operación.
- Estados y eventos históricos.
- Anticipos recibidos.
- Comisiones y retenciones.
- Liquidaciones del factor.
- Confirmación de pagos realizados directamente al factor.
- Alertas por mora del cliente.
- Registro de cobros con recurso contra Extintores Real.
- Integración inicial con movimientos bancarios.

Al finalizar este bloque, el sistema podrá responder:

¿Qué factura fue cedida?
¿A qué empresa de factoring?
¿La operación es con o sin recurso?
¿Cuánto fue cedido?
¿Cuánto debía anticipar el factor?
¿Cuánto fue retenido o descontado?
¿El cliente pagó al factor?
¿El factor todavía debe liquidar fondos?
¿La factura está vencida?
¿El factor inició un cobro con recurso?
¿Qué movimientos bancarios corresponden a la operación?

2. Resumen del bloque

Código	Actividad	Horas	Recurso	Cobertura actual	Resultado
X001	Empresas de factoring	4 h	Backend	Parcial	Empresas, contactos y convenios
X002	Operación de factoring	6 h	Backend	Alta, con ajustes	Cesión y condiciones congeladas
X003	Estados y eventos de factoring	4 h	Backend	Parcial	Máquina de estados e historial
X004	Flujos de anticipo y comisión	8 h	Backend	Alta, con ajustes	Componentes financieros separados
X005	Confirmación de pago al factor	6 h	Backend	Parcial	Pago del cliente sin ingreso bancario propio
X006	Alertas de mora en factoring	4 h	Backend	Parcial	Seguimiento de vencimientos y atrasos
X007	Notificación y cobro con recurso	6 h	Backend	Parcial	Deuda reclamada a Extintores Real

El bloque suma **38 horas**.

La Carta Gantt define:

- X001: catálogo, contactos y condiciones.
- X002: cesión, vencimiento, recurso y montos contratados.
- X003: historial de cesión, mora, pago y liquidación.
- X004: anticipo, comisión, retención y liquidación separados.
- X005: pago del cliente al factor sin ingreso bancario en Extintores Real.
- X006: vencimiento sin pago y seguimiento al cliente.
- X007: registro de la deuda reclamada a Extintores Real.

3. Secuencia funcional

```
X001 Empresa y convenio
→ X002 Cesión de factura
→ X003 Estados e historial
→ X004 Anticipo, comisión, retención y liquidación
→ X005 Confirmación de pago del cliente al factor
→ X006 Seguimiento de mora
→ X007 Cobro con recurso
```

El flujo normal será:

Factura emitida
→ operación de factoring creada
→ factura cedida
→ anticipo recibido
→ cliente paga al factor
→ factor liquida retención o saldo
→ operación cerrada

El flujo de mora con recurso será:

Factura cedida
→ cliente no paga al vencimiento
→ alerta de mora
→ gestión de cobranza
→ factor notifica deuda
→ factor invoca recurso
→ Extintores Real revisa el reclamo
→ pago al factor, si corresponde
→ seguimiento de deuda del cliente

4. Separación de conceptos

El principal riesgo del módulo es representar todo con un único campo `estado`.

Deben distinguirse al menos cuatro dimensiones.

4.1 Operación contractual

Representa:

- Factura cedida.
- Empresa de factoring.
- Fecha de cesión.
- Condición con o sin recurso.
- Monto cedido.
- Condiciones pactadas.

Modelo:

OperacionFactoring

4.2 Estado de cobranza del cliente

Representa si el cliente:

- Todavía debe pagar.
- Pagó parcialmente al factor.
- Pagó completamente al factor.
- Está en mora.

Este estado no implica que Extintores Real haya recibido dinero en su cuenta.

4.3 Obligación financiera del factor

Representa si la empresa de factoring:

- Debe anticipar.
 - Debe liberar una retención.
 - Debe pagar una liquidación.
 - Ya transfirió esos fondos.
 - Todavía mantiene saldo pendiente.
-

4.4 Flujo monetario

Representa un componente económico concreto:

Anticipo
Comisión
Retención
Liquidación
Cobro con recurso
Otro ajuste

Modelo:

FlujoFactoring

5. Cobertura actual del modelo

El archivo actual ya contiene:

EmpresaFactoring
OperacionFactoring

EventoFactoring
FlujoFactoring
PagoCliente
AplicacionPagoFactura
AlertaCobranza

EmpresaFactoring contiene nombre, RUT, correo, teléfono, estado activo y observaciones.

OperacionFactoring se relaciona uno a uno con una factura y ya registra:

- Empresa de factoring.
- Fecha de cesión.
- Fecha de vencimiento.
- Condición con recurso.
- Monto cedido.
- Anticipo.
- Comisión.
- Retención.
- Fechas de pago, liquidación y mora.
- Estado.

EventoFactoring registra acciones como cesión, anticipo, mora, pago al factor, cobro con recurso y liquidación.

FlujoFactoring ya distingue anticipo, liquidación, comisión, retención y cobro con recurso, junto con dirección, fecha, monto y referencia.

PagoCliente ya diferencia entre pagos recibidos por Extintores Real y pagos recibidos por factoring, y puede relacionarse con una operación de factoring.

La base conceptual es adecuada, pero necesita:

1. Contactos múltiples de la empresa de factoring.
2. Convenios o condiciones contractuales.
3. Snapshot de condiciones en cada operación.
4. Validaciones monetarias.
5. Máquina de estados controlada.
6. Eventos más estructurados.
7. Estado de los flujos.
8. Diferenciación entre flujos bancarios y descuentos no bancarios.
9. Confirmación documental del pago al factor.
10. Registro estructurado del cobro con recurso.

6. Decisión sobre la relación factura–factoring

El modelo actual utiliza:

```
factura = models.OneToOneField(...)
```

Esto implica:

Una factura
→ una única operación de factoring

Para el MVP es una decisión razonable cuando:

- Se cede la factura completa.
- No existen cesiones parciales.
- No se cambia de factor.
- No se realizan varias operaciones sobre el mismo documento.

Si el negocio requiere:

- Cesión parcial.
- Cesión de una parte a distintos factores.
- Reemplazo del factor.
- Refinanciamiento de una factura.
- Varias operaciones históricas.

deberá reemplazarse la relación uno a uno por:

FacturaVenta
→ varias CesionFacturaFactoring

con una restricción que impida tener más de una cesión activa sobre el mismo saldo.

Para la primera versión se mantendrá la relación uno a uno, pero se documentará esta limitación.

7. Modelos nuevos y ampliaciones comunes

7.1 Contacto de empresa de factoring

```
class ContactoEmpresaFactoring(ModeloAuditoria):
    class Tipo(models.TextChoices):
        COMERCIAL = "COMERCIAL", "Comercial"
        OPERACIONES = "OPERACIONES", "Operaciones"
        COBRANZA = "COBRANZA", "Cobranza"
        LIQUIDACIONES = "LIQUIDACIONES", "Liquidaciones"
        LEGAL = "LEGAL", "Legal"
        TESORERIA = "TESORERIA", "Tesorería"
        OTRO = "OTRO", "Otro"
```

```

empresa = models.ForeignKey(
    EmpresaFactoring,
    on_delete=models.CASCADE,
    related_name="contactos",
)
nombre = models.CharField(max_length=150)
cargo = models.CharField(max_length=100, blank=True)
tipo = models.CharField(
    max_length=20,
    choices=Tipo.choices,
)
correo = models.EmailField(blank=True)
telefono = models.CharField(max_length=40, blank=True)
principal = models.BooleanField(default=False)
activo = models.BooleanField(default=True)
observaciones = models.TextField(blank=True)

```

7.2 Convenio de factoring

```

class ConvenioFactoring(ModeloAuditoria):
    empresa = models.ForeignKey(
        EmpresaFactoring,
        on_delete=models.PROTECT,
        related_name="convenios",
    )

    nombre = models.CharField(max_length=150)
    referencia_contrato = models.CharField(
        max_length=120,
        blank=True,
    )

    fecha_inicio = models.DateField()
    fecha_fin = models.DateField(
        null=True,
        blank=True,
    )

    moneda = models.CharField(
        max_length=3,
        default="CLP",
    )

    con_recurso_por_defecto = models.BooleanField(
        default=True,
    )

    porcentaje_anticipos = models.DecimalField(

```

```

        max_digits=7,
        decimal_places=4,
        null=True,
        blank=True,
    )

    porcentaje_retencion = models.DecimalField(
        max_digits=7,
        decimal_places=4,
        null=True,
        blank=True,
    )

    porcentaje_comision = models.DecimalField(
        max_digits=7,
        decimal_places=4,
        null=True,
        blank=True,
    )

    dias_liquidacion = models.PositiveIntegerField(
        null=True,
        blank=True,
    )

    dias_gracia = models.PositiveIntegerField(
        default=0,
    )

    activo = models.BooleanField(default=True)
    condiciones = models.JSONField(
        default=dict,
        blank=True,
    )
    observaciones = models.TextField(blank=True)

```

Los porcentajes son condiciones predeterminadas. Cada operación debe guardar una copia de las condiciones efectivamente contratadas.

7.3 Reclamo con recurso

```

class ReclamoRecursoFactoring(ModeloAuditoria):
    class Estado(models.TextChoices):
        NOTIFICADO = "NOTIFICADO", "Notificado"
        EN_REVISION = "EN_REVISION", "En revisión"
        ACEPTADO = "ACEPTADO", "Aceptado"
        OBJETADO = "OBJETADO", "Objetado"
        EN_PAGO = "EN_PAGO", "En proceso de pago"

```



```

PAGADO = "PAGADO", "Pagado"
ANULADO = "ANULADO", "Anulado"

operacion = models.ForeignKey(
    OperacionFactoring,
    on_delete=models.PROTECT,
    related_name="reclamos_recurso",
)

fecha_notificacion = models.DateField()
fecha_limite_respuesta = models.DateField(
    null=True,
    blank=True,
)

monto_reclamado = models.DecimalField(
    max_digits=14,
    decimal_places=2,
)

monto_aceptado = models.DecimalField(
    max_digits=14,
    decimal_places=2,
    default=Decimal("0.00"),
)

referencia = models.CharField(
    max_length=120,
    blank=True,
)

motivo = models.TextField()
fundamento = models.TextField(blank=True)

estado = models.CharField(
    max_length=20,
    choices=Estado.choices,
    default=Estado.NOTIFICADO,
)

respondido_en = models.DateTimeField(
    null=True,
    blank=True,
)

respondido_por = models.ForeignKey(
    settings.AUTH_USER_MODEL,
    on_delete=models.SET_NULL,
    null=True,
    blank=True,
)

```

```
)      related_name="reclamos_factoring_respondidos",
```

X001 — Empresas de factoring

1. Objetivo

Administrar las empresas con las que Extintores Real mantiene o podría mantener operaciones de factoring.

La tarea debe cubrir:

- Identificación.
- Contactos.
- Condiciones.
- Convenios vigentes.
- Estado operativo.
- Historial.

2. Modelos involucrados

Existente:

EmpresaFactoring

Nuevos:

ContactoEmpresaFactoring
ConvenioFactoring

3. Datos de la empresa

Campos actuales:

- Nombre.
- RUT.
- Correo.
- Teléfono.
- Activa.
- Observaciones.

Se recomienda agregar:

```
nombre_fantasia = models.CharField(
    max_length=150,
    blank=True,
)

direccion = models.CharField(
    max_length=250,
    blank=True,
)

sitio_web = models.URLField(blank=True)

codigo_interno = models.CharField(
    max_length=30,
    blank=True,
)

requiere_validacion_manual = models.BooleanField(
    default=False,
)
```

4. Contactos

Una empresa puede tener contactos diferentes para:

- Evaluación comercial.
- Envío de facturas.
- Liquidaciones.
- Cobranza.
- Notificaciones de mora.
- Asuntos legales.
- Tesorería.

No se deben representar todos con el único campo `correo` de la empresa.

El correo y teléfono actuales pueden mantenerse como contacto general.

5. Convenios

Cada convenio debe definir:

- Empresa.
- Referencia contractual.
- Vigencia.
- Moneda.
- Condición con o sin recurso.
- Porcentaje de anticipo.

- Porcentaje de retención.
- Comisión referencial.
- Días esperados para liquidación.
- Días de gracia.
- Condiciones adicionales.
- Documentos adjuntos.

Las condiciones deben provenir del contrato real. El sistema no debe calcular ni asumir cláusulas jurídicas que no estén configuradas.

6. Versionado de convenios

Un convenio utilizado por operaciones históricas no debe modificarse destruyendo sus condiciones originales.

Flujo recomendado:

```
Convenio versión 1
→ se marca inactivo
→ se crea versión 2
```

Cada `OperacionFactoring` guarda:

- Convenio.
 - Versión.
 - Snapshot de las condiciones.
-

7. Reglas de negocio

1. El nombre de la empresa es obligatorio.
 2. El RUT se normaliza y valida.
 3. No pueden existir empresas activas con el mismo RUT.
 4. Una empresa inactiva no se utiliza en operaciones nuevas.
 5. Las operaciones históricas mantienen su empresa.
 6. Un contacto inactivo no aparece en operaciones nuevas.
 7. Solo debe existir un contacto principal activo por tipo.
 8. Un convenio vencido no se selecciona para nuevas operaciones.
 9. Una empresa con operaciones activas no puede eliminarse.
 10. Los cambios de convenio no modifican operaciones existentes.
 11. La moneda de la operación debe ser compatible con el convenio.
 12. Las condiciones predeterminadas pueden modificarse al crear una operación, pero la desviación debe quedar registrada.
-

8. URLs

```
/administracion/factoring/empresas/  
/administracion/factoring/empresas/nueva/  
/administracion/factoring/empresas/<pk>/  
/administracion/factoring/empresas/<pk>/editar/  
/administracion/factoring/empresas/<pk>/activar/  
/administracion/factoring/empresas/<pk>/desactivar/  
  
/administracion/factoring/empresas/<pk>/contactos/  
/administracion/factoring/contactos/nuevo/  
/administracion/factoring/contactos/<pk>/editar/  
  
/administracion/factoring/empresas/<pk>/convenios/  
/administracion/factoring/convenios/nuevo/  
/administracion/factoring/convenios/<pk>/  
/administracion/factoring/convenios/<pk>/versionar/
```

9. Vistas

```
EmpresaFactoringListView  
EmpresaFactoringDetailView  
EmpresaFactoringCreateView  
EmpresaFactoringUpdateView  
EmpresaFactoringActivarView  
EmpresaFactoringDesactivarView  
  
ContactoFactoringCreateView  
ContactoFactoringUpdateView  
ContactoFactoringDesactivarView  
  
ConvenioFactoringListView  
ConvenioFactoringCreateView  
ConvenioFactoringDetailView  
ConvenioFactoringVersionarView
```

10. Formularios

```
EmpresaFactoringForm  
ContactoEmpresaFactoringForm  
ConvenioFactoringForm
```

El formulario del convenio debe validar:

- Porcentajes entre 0 y 100.
 - Fecha final posterior a fecha inicial.
 - Días no negativos.
 - Empresa activa.
 - Moneda admitida.
 - Referencia contractual.
-

11. Permisos

```
view_empresafactoring
add_empresafactoring
change_empresafactoring
desactivar_empresafactoring

view_contactoempresafactoring
add_contactoempresafactoring
change_contactoempresafactoring

view_conveniofactoring
add_conveniofactoring
versionar_conveniofactoring
```

12. Auditoría

Registrar:

- Creación de empresa.
 - Cambio de RUT.
 - Activación y desactivación.
 - Alta y baja de contactos.
 - Cambio de condiciones.
 - Nueva versión de convenio.
 - Adjuntos contractuales.
-

13. Pruebas

- Crear empresa.
- RUT duplicado.
- Desactivar empresa sin operaciones.
- Impedir usar empresa inactiva.
- Varios contactos.
- Contacto principal.
- Convenio vigente.

- Convenio vencido.
 - Porcentaje inválido.
 - Nueva versión.
 - Conservación histórica.
 - Permisos.
-

14. Criterios de aceptación

- Existe un catálogo de empresas.
 - Se administran contactos especializados.
 - Se registran convenios.
 - Las condiciones están versionadas.
 - Las empresas inactivas no se usan en nuevas operaciones.
 - Las operaciones históricas conservan sus condiciones.
 - Los cambios quedan auditados.
-

15. Evaluación de esfuerzo

Las **4 horas** de la Gantt son suficientes para ampliar el CRUD básico de `EmpresaFactoring`.

La construcción completa de contactos, convenios, versionado y pruebas probablemente requiere distribuir trabajo adicional dentro de X002 y X003.

X002 — Operación de factoring

1. Objetivo

Registrar la cesión de una factura y congelar las condiciones particulares de la operación.

La operación debe indicar:

- Factura.
 - Empresa de factoring.
 - Convenio.
 - Fecha de cesión.
 - Vencimiento.
 - Condición con o sin recurso.
 - Monto cedido.
 - Anticipo.
 - Comisión.
 - Retención.
 - Referencias contractuales.
 - Estado inicial.
-

2. Modelo principal

OperacionFactoring

El modelo actual ya contiene la relación con factura y empresa, fechas, recurso y componentes monetarios.

3. Campos adicionales recomendados

```
convenio = models.ForeignKey(
    ConvenioFactoring,
    on_delete=models.PROTECT,
    null=True,
    blank=True,
    related_name="operaciones",
)

referencia_operacion = models.CharField(
    max_length=120,
    blank=True,
)

moneda = models.CharField(
    max_length=3,
    default="CLP",
)

cliente_notificado_en = models.DateTimeField(
    null=True,
    blank=True,
)

notificacion_cliente_referencia = models.CharField(
    max_length=120,
    blank=True,
)

porcentaje_anticipo_pactado = models.DecimalField(
    max_digits=7,
    decimal_places=4,
    null=True,
    blank=True,
)

porcentaje_comision_pactado = models.DecimalField(
    max_digits=7,
    decimal_places=4,
```



```

        null=True,
        blank=True,
    )

    porcentaje_retencion_pactado = models.DecimalField(
        max_digits=7,
        decimal_places=4,
        null=True,
        blank=True,
    )

    condiciones_snapshot = models.JSONField(
        default=dict,
        blank=True,
    )

    fecha_estimada_anticipo = models.DateField(
        null=True,
        blank=True,
    )

    fecha_estimada_liquidacion = models.DateField(
        null=True,
        blank=True,
    )

```

4. Facturas elegibles

Solo podrán cederse facturas:

- Emitidas.
- No anuladas.
- Con saldo pendiente.
- Del cliente indicado.
- Sin otra operación activa.
- Con moneda compatible.
- Sin bloqueo administrativo.
- Con documentos requeridos.

FacturaVenta distingue facturas emitidas y anuladas, y calcula su saldo pendiente a partir de las aplicaciones válidas.

5. Monto cedible

La regla inicial será:

monto cedido
≤ saldo pendiente de la factura

Cuando la política solo permita cesión total:

monto cedido
= saldo pendiente de la factura

La modalidad parcial debe configurarse explícitamente.

6. Validaciones monetarias

1. `monto_cedido > 0`.
2. `monto_cedido ≤ saldo_cedible`.
3. `monto_anticipo ≥ 0`.
4. `monto_anticipo ≤ monto_cedido`.
5. `comision ≥ 0`.
6. `retencion ≥ 0`.
7. `retencion ≤ monto_cedido`.
8. `monto_anticipo + retencion ≤ monto_cedido`, salvo configuración contractual documentada.
9. Los porcentajes deben coincidir razonablemente con los montos.
10. Las diferencias de cálculo requieren observación.

No se debe imponer una fórmula universal del tipo:

`anticipo + comisión + retención = monto cedido`

porque la comisión puede:

- Descontarse del anticipo.
- Descontarse de la liquidación.
- Facturarse separadamente.
- Incorporar impuestos.
- Tener mínimos fijos.

La fórmula se determinará según el convenio.

7. Snapshot contractual

Al crear la operación se copiarán:

- Empresa.
- Convenio.
- Recurso.

- Porcentajes.
- Días de liquidación.
- Moneda.
- Referencia.
- Condiciones especiales.

Una modificación posterior del convenio no cambia la operación.

8. Estado inicial

La operación se crea como:

CEDIDA

Simultáneamente:

- Se crea `EventoFactoring.FACTURA_CEDIDA`.
- Se actualiza la factura a `CEDIDA_FACTORING`.
- Se generan flujos esperados.
- Se registra auditoría.

Mientras una operación permanezca activa, el cálculo actual de la factura la considera cedida a factoring.

9. Servicio

services/factoring/operaciones.py

```
@transaction.atomic
def crear_operacion_factoring(
    *,
    factura_id,
    empresa_id,
    convenio_id,
    fecha_cesion,
    monto_cedido,
    monto_anticipo,
    comision,
    retencion,
    con_recurso,
    usuario,
    observaciones="",
):
    factura = (
        FacturaVenta.objects
```

```

        .select_for_update()
        .get(pk=factura_id)
    )

    # Validar factura.
    # Validar convenio.
    # Calcular saldo cedible.
    # Congelar condiciones.
    # Crear operación.
    # Crear evento.
    # Generar flujos esperados.
    # Actualizar estado de factura.
    # Registrar auditoría.

```

Funciones:

```

crear_operacion_factoring()
modificar_operacion_cedida()
anular_operacion_factoring()
calcular_monto_cedible()
generar_snapshot_condiciones()

```

10. Edición

La operación podrá editarse libremente solo antes de que exista:

- Anticipo ejecutado.
- Pago del cliente al factor.
- Movimiento bancario conciliado.
- Notificación de mora.
- Reclamo con recurso.

Después deberá utilizarse una corrección o reversa controlada.

11. Anulación

La operación no se elimina.

Para anular:

1. La factura debe seguir válida.
2. No deben existir flujos conciliados, o deben reversarse.
3. No debe existir pago confirmado al factor.
4. No debe existir reclamo activo.
5. Se requiere motivo.
6. Se crea evento de anulación.

7. Se recalcula el estado de la factura.

12. URLs

```
/administracion/factoring/operaciones/  
/administracion/factoring/operaciones/nueva/  
/administracion/factoring/operaciones/<pk>/  
/administracion/factoring/operaciones/<pk>/editar/  
/administracion/factoring/operaciones/<pk>/anular/  
/administracion/facturas/<factura_id>/ceder/
```

13. Vistas

```
OperacionFactoringListView  
OperacionFactoringDetailView  
OperacionFactoringCreateView  
OperacionFactoringUpdateView  
OperacionFactoringAnularView  
FacturaCederFactoringView
```

14. Permisos

```
view_operacionfactoring  
add_operacionfactoring  
change_operacionfactoring  
anular_operacionfactoring  
ceder_factura_factoring
```

15. Auditoría

Registrar:

- Factura.
- Saldo cedible.
- Monto cedido.
- Condiciones.
- Convenio.
- Usuario.
- Fechas.
- Estado anterior y nuevo.

- Snapshot.
-

16. Pruebas

- Cesión total.
 - Cesión parcial autorizada.
 - Factura pagada.
 - Factura anulada.
 - Factura con operación activa.
 - Monto superior al saldo.
 - Anticipo superior.
 - Retención inválida.
 - Convenio vencido.
 - Moneda incompatible.
 - Snapshot.
 - Anulación.
 - Concurrencia.
-

17. Criterios de aceptación

- Se puede crear una operación válida.
 - La factura queda cedida.
 - Las condiciones quedan congeladas.
 - No se cede más que el saldo.
 - No existen dos operaciones activas sobre la misma factura.
 - Los flujos esperados quedan preparados.
 - La creación es transaccional.
 - La actividad queda auditada.
-

X003 — Estados y eventos de factoring

1. Objetivo

Controlar las transiciones de la operación y conservar un historial inmutable.

El estado no debe editarse directamente desde un formulario general.

2. Estados existentes

El modelo actual contiene:

CEDIDA
ANTICIPADA

```
POR_COBRAR_CLIENTE
CLIENTE_PAGO_FACTOR
FACTOR_DEBE_LIQUIDAR
LIQUIDADA
EN_MORA
COBRO_RECURSO
CERRADA
ANULADA
```

Estos estados cubren el proceso, pero mezclan:

- Situación contractual.
- Cobranza.
- Obligación del factor.
- Cierre financiero.

Para el MVP se conservarán, pero todas las transiciones serán controladas por servicios.

3. Flujo normal

```
CEDIDA
→ ANTICIPADA
→ POR_COBRAR_CLIENTE
→ CLIENTE_PAGO_FACTOR
→ FACTOR_DEBE_LIQUIDAR
→ LIQUIDADA
→ CERRADA
```

La transición `ANTICIPADA` debe ocurrir cuando el anticipo haya sido efectivamente ejecutado según la política definida, no solo porque exista un monto pactado.

4. Flujo sin anticipo

Cuando el convenio no contempla anticipo:

```
CEDIDA
→ POR_COBRAR_CLIENTE
```

5. Flujo de mora

```
POR_COBRAR_CLIENTE
→ EN_MORA
```

La mora no debe activarse antes de:

fecha de vencimiento + días de gracia

6. Flujo con recurso

EN_MORA
→ COBRO_RECUSO

Solo se permite automáticamente cuando:

con_recurso = True

Una reclamación sobre una operación sin recurso debe marcarse como excepción contractual y requerir revisión especial.

7. Cierre

La operación podrá cerrarse cuando:

1. La factura fue pagada o regularizada.
2. Los flujos del factor están completados.
3. Las comisiones y retenciones están registradas.
4. Los flujos bancarios requeridos están conciliados.
5. No existen reclamos abiertos.
6. No existen alertas críticas.
7. No existen saldos pendientes.

8. Ampliación de EventoFactoring

El modelo actual conserva operación, tipo, fecha, detalle y usuario.

Se recomienda agregar:

```
estado_anterior = models.CharField(  
    max_length=30,  
    blank=True,  
)  
  
estado_nuevo = models.CharField(  
    max_length=30,
```



```

        blank=True,
    )

    monto = models.DecimalField(
        max_digits=14,
        decimal_places=2,
        null=True,
        blank=True,
    )

    referencia = models.CharField(
        max_length=120,
        blank=True,
    )

    datos = models.JSONField(
        default=dict,
        blank=True,
    )

    correlation_id = models.UUIDField(
        default=uuid.uuid4,
        editable=False,
    )

```

También deben agregarse tipos:

```

CLIENTE_NOTIFICADO
ANTICIPO_PENDIENTE
ANTICIPO_CONCILIADO
PAGO_PARCIAL_FACTOR
FACTOR_LIQUIDO_PARCIAL
COMISION_REGISTRADA
RETENCION_REGISTRADA
RECLAMO_OBJETADO
OPERACION_CERRADA
OPERACION_ANULADA

```

9. Inmutabilidad

Los eventos:

- No se editan desde la interfaz.
- No se eliminan.
- Se corrigen mediante un nuevo evento.
- Conservan usuario y fecha.
- Incluyen el estado anterior y nuevo.
- Incluyen los montos relevantes.

10. Servicio de transiciones

```
services/factoring/workflow.py
```

```
@transaction.atomic
def cambiar_estado_operacion(
    *,
    operacion_id,
    estado_nuevo,
    usuario,
    tipo_evento,
    detalle="",
    datos=None,
):
    operacion = (
        OperacionFactoring.objects
        .select_for_update()
        .get(pk=operacion_id)
    )

    # Validar transición.
    # Validar condiciones financieras.
    # Actualizar estado.
    # Crear evento.
    # Actualizar factura.
    # Actualizar alertas.
```

11. Matriz de transición

Estado actual	Estado nuevo	Permitido
CEDIDA	ANTICIPADA	Sí, con anticipo ejecutado
CEDIDA	POR_COBRAR_CLIENTE	Sí, sin anticipo
ANTICIPADA	POR_COBRAR_CLIENTE	Sí
POR_COBRAR_CLIENTE	CLIENTE_PAGO_FACTOR	Sí
POR_COBRAR_CLIENTE	EN_MORA	Sí, vencida
EN_MORA	CLIENTE_PAGO_FACTOR	Sí
EN_MORA	COBRO_RECURSO	Sí, con recurso
CLIENTE_PAGO_FACTOR	FACTOR_DEBE_LIQUIDAR	Sí
FACTOR_DEBE_LIQUIDAR	LIQUIDADA	Sí

Estado actual	Estado nuevo	Permitido
LIQUIDADA	CERRADA	Sí
CERRADA	CEDIDA	No, requiere reapertura
ANULADA	Cualquier estado	No

12. Interfaz de historial

URL:

```
/administracion/factoring/operaciones/<pk>/historial/
```

Línea de tiempo:

```
Factura cedida
→ anticipo recibido
→ cliente notificado
→ cliente en mora
→ cliente pagó al factor
→ factor liquidó
→ operación cerrada
```

13. Permisos

```
view_eventofactoring
registrar_eventofactoring
cambiar_estado_operacionfactoring
reabrir_operacionfactoring
```

14. Auditoría

`EventoFactoring` constituye el historial de dominio.

`EventoAuditoria` deberá conservar además:

- Petición.
- IP.
- Usuario.
- Valores anteriores.
- Valores nuevos.

- Objeto afectado.
-

15. Pruebas

- Flujo normal.
 - Sin anticipo.
 - Mora.
 - Pago después de mora.
 - Recurso.
 - Transición inválida.
 - Cierre con flujos pendientes.
 - Evento inmutable.
 - Dos cambios simultáneos.
 - Reapertura.
 - Auditoría.
-

16. Criterios de aceptación

- El estado no se modifica directamente.
 - Solo se permiten transiciones válidas.
 - Cada transición crea un evento.
 - Los eventos no se sobrescriben.
 - Los estados financieros se validan.
 - El historial muestra el proceso completo.
 - Las operaciones concurrentes son seguras.
-

X004 — Flujos de anticipo y comisión

1. Objetivo

Registrar separadamente todos los componentes económicos de una operación de factoring.

La Carta Gantt exige separar:

Anticipo
Comisión
Retención
Liquidación

2. Modelo principal

FlujoFactoring

Los tipos actuales ya incluyen anticipo, liquidación, comisión, retención y cobro con recurso.

3. Problema de la dirección actual

Actualmente cada flujo exige:

INGRESO
o
EGRESO

Sin embargo:

- Una comisión puede descontarse del anticipo sin producir un egreso bancario separado.
- Una retención representa dinero retenido por el factor, no necesariamente un movimiento bancario.
- Una compensación puede reducir una liquidación sin aparecer como una transacción individual.

Por tanto, debe agregarse una naturaleza financiera.

4. Naturaleza propuesta

```
class Naturaleza(models.TextChoices):  
    MOVIMIENTO_BANCARIO = (  
        "MOVIMIENTO_BANCARIO",  
        "Movimiento bancario",  
    )  
    DESCUENTO = "DESCUENTO", "Descuento"  
    RETENCION = "RETENCION", "Retención"  
    COMPENSACION = "COMPENSACION", "Compensación"  
    AJUSTE = "AJUSTE", "Ajuste"
```

Campos:

```
naturaleza = models.CharField(  
    max_length=30,  
    choices=Naturaleza.choices,  
    default=Naturaleza.MOVIMIENTO_BANCARIO,  
)  
  
requiere_conciliacion_bancaria = models.BooleanField(  
    default=False,  
    help_text="Indica si se requiere conciliación bancaria"
```

```
        default=True,  
    )
```

La dirección será obligatoria únicamente cuando el flujo requiera movimiento bancario.

5. Estado del flujo

```
class Estado(models.TextChoices):  
    ESPERADO = "ESPERADO", "Esperado"  
    INFORMADO = "INFORMADO", "Informado"  
    PARCIAL = "PARCIAL", "Parcialmente ejecutado"  
    EJECUTADO = "EJECUTADO", "Ejecutado"  
    CONCILIADO = "CONCILIADO", "Conciliado"  
    ANULADO = "ANULADO", "Anulado"
```

Campos adicionales:

```
estado = models.CharField(  
    max_length=20,  
    choices=Estado.choices,  
    default=Estado.ESPERADO,  
)  
  
fecha_esperada = models.DateField(  
    null=True,  
    blank=True,  
)  
  
fecha_ejecucion = models.DateField(  
    null=True,  
    blank=True,  
)  
  
anulado_en = models.DateTimeField(  
    null=True,  
    blank=True,  
)  
  
anulado_por = models.ForeignKey(  
    settings.AUTH_USER_MODEL,  
    on_delete=models.SET_NULL,  
    null=True,  
    blank=True,  
    related_name="flujos_factoring anulados",  
)
```

```
motivo_anulacion = models.TextField(blank=True)
```

6. Matriz de flujos

Tipo	Naturaleza habitual	Dirección bancaria
Anticipo	Movimiento bancario	Ingreso
Comisión descontada	Descuento	No aplica
Comisión pagada separadamente	Movimiento bancario	Egreso
Retención	Retención	No aplica
Liquidación	Movimiento bancario	Ingreso
Cobro con recurso	Movimiento bancario	Egreso
Ajuste	Configurable	Según caso

La palabra “habitual” es importante: el convenio puede definir un tratamiento diferente.

7. Ejemplo de operación

Monto cedido:	1.000.000 CLP
Anticipo contractual:	800.000 CLP
Comisión:	20.000 CLP
Retención:	180.000 CLP

Una estructura posible sería:

ANTICIPO	
Naturaleza:	movimiento bancario
Ingreso esperado:	780.000 CLP
COMISIÓN	
Naturaleza:	descuento
Monto:	20.000 CLP
RETENCIÓN	
Naturaleza:	retención
Monto:	180.000 CLP

Este ejemplo corresponde a una comisión descontada del anticipo. Otro convenio puede descontarla de la liquidación o cobrarla separadamente.

El sistema debe usar las condiciones configuradas, no asumir esta estructura para todas las operaciones.

8. Generación inicial

Al crear la operación se generarán flujos esperados según el convenio:

```
Anticipo esperado
Comisión esperada
Retención esperada
Liquidación esperada
```

Los flujos esperados no equivalen a dinero recibido.

9. Conciliación bancaria

Para un flujo bancario:

```
MovimientoBancario
→ AplicacionMovimientoBancario
→ FlujoFactoring
```

La dirección debe coincidir con `FlujoFactoring.direccion`.

El puente bancario actual ya admite `FlujoFactoring` como destino; el flujo define si corresponde a ingreso o egreso. La validación debe ejecutarse en el servicio transaccional.

10. Saldos del flujo

```
@property
def monto_conciliado(self):
    total = self.aplicaciones_bancarias.filter(
        activa=True
    ).aggregate(total=Sum("monto"))["total"]

    return total or Decimal("0.00")
```

```
@property
def saldo_por_conciliar(self):
    if not self.requiere_conciliacion_bancaria:
        return Decimal("0.00")
```



```
return self.monto - self.monto_conciliado
```

11. Reglas

1. El monto debe ser positivo.
2. La operación no puede estar anulada.
3. El tipo debe ser compatible con la naturaleza.
4. Los flujos bancarios requieren dirección.
5. Los descuentos y retenciones no deben crear movimientos bancarios ficticios.
6. Un flujo conciliado no se edita.
7. Un flujo anulado no suma.
8. La suma de componentes debe ser revisable contra el monto cedido.
9. Las diferencias requieren explicación.
10. Los flujos pueden ser parciales.
11. No se supera el monto del flujo.
12. No se supera el movimiento bancario.

12. URLs

```
/administracion/factoring/operaciones/<pk>/flujos/  
/administracion/factoring/operaciones/<pk>/flujos/nuevo/  
/administracion/factoring/flujos/<pk>/  
/administracion/factoring/flujos/<pk>/editar/  
/administracion/factoring/flujos/<pk>/ejecutar/  
/administracion/factoring/flujos/<pk>/conciliar/  
/administracion/factoring/flujos/<pk>/anular/
```

13. Vistas

```
FlujoFactoringListView  
FlujoFactoringCreateView  
FlujoFactoringDetailView  
FlujoFactoringUpdateView  
FlujoFactoringEjecutarView  
FlujoFactoringConciliarView  
FlujoFactoringAnularView
```

14. Servicio

```
services/factoring/flujos.py
```

Funciones:

```
generar_flujos_esperados()  
registrar_flujo()  
ejecutar_flujo()  
conciliar_flujo()  
anular_flujo()  
calcular_resumen_operacion()  
actualizar_estado_operacion()
```

15. Pantalla de resumen

Componente	Esperado	Ejecutado	Conciliado	Pendiente
Anticipo	780.000 CLP	780.000 CLP	780.000 CLP	0 CLP
Comisión	20.000 CLP	20.000 CLP	No aplica	0 CLP
Retención	180.000 CLP	180.000 CLP	No aplica	0 CLP
Liquidación	180.000 CLP	0 CLP	0 CLP	180.000 CLP

16. Permisos

```
view_flujofactoring  
add_flujofactoring  
change_flujofactoring  
ejecutar_flujofactoring  
conciliar_flujofactoring  
anular_flujofactoring
```

17. Auditoría

Registrar:

- Tipo.
- Naturaleza.
- Monto.
- Estado.

- Fecha esperada.
 - Fecha efectiva.
 - Movimiento bancario.
 - Usuario.
 - Anulación.
-

18. Pruebas

- Anticipo ingreso.
 - Comisión descontada.
 - Comisión pagada.
 - Retención.
 - Liquidación.
 - Flujo parcial.
 - Conciliación completa.
 - Dirección incorrecta.
 - Flujo no bancario.
 - Exceso sobre flujo.
 - Exceso sobre movimiento.
 - Anulación.
 - Actualización de operación.
-

19. Criterios de aceptación

- Los componentes están separados.
 - No se crean movimientos bancarios artificiales.
 - Los flujos esperados se distinguen de los ejecutados.
 - Las conciliaciones parciales funcionan.
 - Los saldos se calculan correctamente.
 - El estado de la operación se actualiza.
 - Existe trazabilidad completa.
-

X005 — Confirmación de pago al factor

1. Objetivo

Registrar que el cliente pagó directamente a la empresa de factoring, aunque no exista ingreso bancario en una cuenta de Extintores Real.

La Carta Gantt especifica expresamente que este pago no produce un ingreso bancario propio.

2. Modelos utilizados

```
PagoCliente  
AplicacionPagoFactura  
EventoFactoring  
OperacionFactoring
```

`PagoCliente` ya admite:

```
receptor = FACTORING
```

y una relación con `OperacionFactoring`.

3. Principio fundamental

El flujo correcto es:

```
Cliente paga al factor  
→ PagoCliente receptor FACTORING  
→ AplicacionPagoFactura  
→ EventoFactoring CLIENTE_PAGO_FACTOR
```

No debe crearse:

```
AplicacionMovimientoBancario
```

contra una cuenta de Extintores Real, porque ese dinero no ingresó a su banco.

4. Evidencia de confirmación

El pago puede confirmarse mediante:

- Informe del factor.
- Comprobante enviado por el cliente.
- Portal del factor.
- Correo de confirmación.
- Archivo de liquidación.
- Documento contractual.

Se recomienda registrar en el evento:

```
fuente_confirmacion = models.CharField(  
    max_length=30,  
    blank=True,  
)  
  
documento_referencia = models.CharField(  
    max_length=120,  
    blank=True,  
)
```

Los archivos se gestionarán mediante el modelo transversal de adjuntos.

5. Pagos parciales

Debe permitirse:

Factura cedida:	1.000.000 CLP
Pago al factor 1:	600.000 CLP
Pago al factor 2:	400.000 CLP

Después del primer pago:

Cliente pagó parcialmente al factor
Saldo del cliente: 400.000 CLP

No debe cambiarse a `CLIENTE_PAGO_FACTOR` completo hasta que el monto válido alcance la obligación cedida.

Puede agregarse un estado:

`CLIENTE_PAGO_PARCIAL_FACTOR`

o calcularse como una condición complementaria.

6. Aplicación a la factura

`AplicacionPagoFactura` ya permite aplicar un pago a una factura y evita superar el pago o el total de la factura.

Para un pago recibido por factoring:

estado = APLICADA

si:

- La operación corresponde a esa factura.
- El cliente coincide.
- El pago fue confirmado.
- No existe una irregularidad.

Esto reduce el saldo del cliente, pero no significa que el factor haya liquidado todo lo adeudado a Extintores Real.

7. Estado bancario especial

Para estos pagos debe mostrarse:

Conciliación bancaria propia: NO APLICA

No:

No conciliado

porque “no conciliado” sugeriría que falta encontrar un depósito propio.

Se recomienda incorporar un estado calculado:

NO_REQUIERE_CONCILIACION

cuando:

PagoCliente.receptor = FACTORING

8. Consecuencia sobre la operación

Pago parcial

- Se registra evento.
- Se actualiza saldo del cliente.
- Se mantiene cobranza parcial.
- No se activa todavía la liquidación final completa.

Pago total

```
OperacionFactoring.estado  
→ CLIENTE_PAGO_FACTOR  
→ FACTOR_DEBE_LIQUIDAR
```

Se activa o actualiza el flujo de liquidación del factor.

9. Servicio

```
services/factoring/pagos_cliente.py
```

```
@transaction.atomic  
def confirmar_pago_cliente_factor(  
    *,  
    operacion_id,  
    fecha,  
    monto,  
    forma_pago,  
    referencia,  
    fuente,  
    usuario,  
    observaciones="",  
):  
    operacion = (  
        OperacionFactoring.objects  
        .select_for_update()  
        .select_related("factura_cliente")  
        .get(pk=operacion_id)  
    )  
  
    # Validar operación.  
    # Validar cliente y saldo.  
    # Crear PagoCliente receptor FACTORING.  
    # Crear AplicacionPagoFactura.  
    # Crear evento.  
    # Actualizar factura.  
    # Actualizar operación.  
    # Crear o ajustar flujo de liquidación.
```

10. Reglas

1. La operación debe estar activa.
2. El receptor debe ser FACTORING.

3. El pago debe pertenecer al cliente de la factura.
4. La operación relacionada es obligatoria.
5. El monto debe ser positivo.
6. No se supera el saldo del cliente.
7. No se supera el monto cedido.
8. La fecha no puede ser anterior a la cesión.
9. La evidencia debe quedar identificada.
10. No se crea movimiento bancario propio.
11. El pago puede ser parcial.
12. La confirmación puede reversarse con motivo.
13. La reversa actualiza factura, operación y flujos.
14. No se confunde pago del cliente con liquidación del factor.

11. URLs

```
/administracion/factoring/operaciones/<pk>/confirmar-pago-cliente/  
/administracion/factoring/operaciones/<pk>/pagos-cliente/  
/administracion/factoring/pagos-cliente/<pk>/  
/administracion/factoring/pagos-cliente/<pk>/reversar/
```

12. Vistas

```
PagoClienteFactorConfirmarView  
PagoClienteFactorListView  
PagoClienteFactorDetailView  
PagoClienteFactorReversarView
```

13. Interfaz

Mostrar:

```
Factura: 1005  
Monto cedido:          1.000.000 CLP  
Pagado al factor:      600.000 CLP  
Saldo del cliente:     400.000 CLP  
Conciliación bancaria ER: No aplica
```


14. Permisos

```
confirmar_pago_cliente_factor  
reversar_pago_cliente_factor  
view_pago_cliente_factor
```

15. Auditoría

Registrar:

- Operación.
 - Factura.
 - Cliente.
 - Monto.
 - Fuente.
 - Referencia.
 - Usuario.
 - Evidencia.
 - Estado anterior y nuevo.
-

16. Pruebas

- Pago completo.
 - Pago parcial.
 - Varios pagos.
 - Monto superior.
 - Cliente incorrecto.
 - Operación anulada.
 - Fecha anterior a cesión.
 - Sin movimiento bancario propio.
 - Creación de flujo de liquidación.
 - Reversa.
 - Concurrencia.
-

17. Criterios de aceptación

- Se confirma el pago al factor.
- El pago reduce la deuda del cliente.
- No se crea un ingreso bancario ficticio.
- Se soportan pagos parciales.
- La obligación del factor permanece separada.
- La operación cambia de estado correctamente.
- La evidencia queda disponible.
- La acción queda auditada.

X006 — Alertas de mora en factoring

1. Objetivo

Detectar operaciones donde la factura cedida llegó a vencimiento y el cliente todavía no ha pagado al factor.

La Carta Gantt exige controlar el vencimiento sin pago y el seguimiento al cliente.

2. Modelo involucrado

AlertaCobranza

El modelo actual ya incluye:

FACTORING_VENCIDO
FACTOR_NOTIFICA_DEUDA

y puede relacionarse con una operación de factoring.

3. Tipos adicionales recomendados

```
CLIENTE_MORA_FACTORING = (  
    "CLIENTE_MORA_FACTORING",  
    "Cliente en mora con factoring",  
)  
  
PAGO_PARCIAL_FACTORING_VENCIDO = (  
    "PAGO_PARCIAL_FACTORING_VENCIDO",  
    "Pago parcial vencido en factoring",  
)  
  
FACTOR_SIN_LIQUIDAR = (  
    "FACTOR_SIN_LIQUIDAR",  
    "Factor pendiente de liquidación",  
)  
  
ANTICIPO_FACTORING_ATRASADO = (  
    "ANTICIPO_FACTORING_ATRASADO",  
    "Anticipo de factoring atrasado",  
)
```

Agregar además:

```
clave_control = models.CharField(
    max_length=200,
    unique=True,
)

fecha_limite = models.DateField(
    null=True,
    blank=True,
)

asignada_a = models.ForeignKey(
    settings.AUTH_USER_MODEL,
    on_delete=models.SET_NULL,
    null=True,
    blank=True,
    related_name="alertas_factoring_asignadas",
)

motivo_resolucion = models.TextField(blank=True)
```

4. Condición de mora

Una operación estará en mora cuando:

```
fecha actual
>
fecha de vencimiento + días de gracia
```

y:

```
monto pagado al factor
<
monto que debía pagar el cliente
```

No se debe considerar únicamente el estado guardado; el servicio debe recalcular la condición.

5. Monto vencido

```
monto vencido
=
monto cedido
```

- pagos confirmados al factor
- notas de crédito o ajustes válidos

No deben descontarse:

- Anticipos recibidos por Extintores Real.
- Comisiones.
- Retenciones.
- Liquidaciones del factor.

Esos conceptos no son pagos del cliente sobre la factura.

6. Severidad

Antigüedad	Severidad
1-5 días	Media
6-15 días	Alta
Más de 15 días	Crítica
Factor invoca recurso	Crítica

La severidad también puede considerar:

- Monto.
- Cliente.
- Condición con recurso.
- Historial de mora.
- Proximidad de reclamación contractual.

7. Acciones de seguimiento

Desde la alerta se podrá:

- Abrir la operación.
- Abrir la factura.
- Registrar gestión de cobranza.
- Contactar al cliente.
- Registrar respuesta.
- Confirmar pago al factor.
- Registrar notificación del factor.
- Escalar a revisión de recurso.

Las gestiones de cobranza existentes pueden seguir vinculándose a la factura.

8. Resolución automática

La alerta se resuelve cuando:

- El cliente paga completamente al factor.
- La operación se anula.
- La deuda se corrige mediante nota de crédito.
- Se registra una regularización autorizada.
- Se transforma en reclamo con recurso, cerrando la alerta de mora y abriendo la correspondiente al reclamo.

9. Prevención de duplicados

Ejemplo de clave:

```
CLIENTE_MORA_FACTORING:operacion:154
```

La ejecución diaria no debe generar una nueva alerta mientras exista una abierta o en gestión.

10. Comando

```
python manage.py generar_alertas_factoring
```

Flujo:

```
Buscar operaciones activas
→ recalcular pagos al factor
→ evaluar vencimiento
→ crear alertas
→ actualizar severidad
→ resolver alertas corregidas
→ detectar factor sin liquidar
```

11. URLs

```
/administracion/factoring/alertas/
/administracion/factoring/alertas/<pk>/
/administracion/factoring/alertas/<pk>/gestionar/
/administracion/factoring/alertas/<pk>/resolver/
```

12. Vistas

```
AlertasFactoringListView  
AlertaFactoringDetailView  
AlertaFactoringGestionarView  
AlertaFactoringResolverView
```

13. Servicio

```
services/factoring/alertas.py
```

Funciones:

```
generar_alertas_mora()  
generar_alertas_anticipo()  
generar_alertas_liquidacion()  
actualizar_severidad()  
resolver_alertas_corregidas()  
asignar_alerta()  
resolver_alerta()
```

14. Permisos

```
view_alertas_factoring  
gestionar_alertas_factoring  
resolver_alertas_factoring
```

15. Auditoría

Registrar:

- Creación.
- Días de mora.
- Monto vencido.
- Severidad.
- Asignación.
- Gestiones.
- Resolución.
- Transformación en reclamo con recurso.

16. Pruebas

- Factura no vencida.
 - Primer día de mora.
 - Período de gracia.
 - Pago parcial.
 - Pago completo.
 - Operación anulada.
 - Con recurso.
 - Sin recurso.
 - Cambio de severidad.
 - Idempotencia.
 - Resolución automática.
 - Factor pendiente de liquidar.
-

17. Criterios de aceptación

- Las operaciones vencidas generan alerta.
 - Se calcula el monto vencido.
 - Los pagos parciales se consideran.
 - Las alertas no se duplican.
 - La severidad aumenta con el atraso.
 - Se puede registrar seguimiento.
 - Las condiciones corregidas resuelven la alerta.
 - Existe trazabilidad.
-

18. Evaluación de esfuerzo

Las **4 horas** permiten una primera regla de `FACTORING_VENCIDO` y una bandeja básica.

El escalamiento, la liquidación atrasada, los plazos configurables y las pruebas deben continuar en X008–X013.

X007 — Notificación y cobro con recurso

1. Objetivo

Registrar formalmente que la empresa de factoring reclama a Extintores Real el pago de una deuda bajo una operación con recurso.

La tarea no debe limitarse a cambiar:

```
OperacionFactoring.estado = COBRO_RECURSO
```

Debe conservar:

- Notificación.
- Monto reclamado.
- Fecha límite.
- Motivo.
- Documentos.
- Revisión.
- Respuesta.
- Monto aceptado.
- Pago.
- Conciliación.
- Recuperación posterior frente al cliente.

2. Modelos involucrados

Existentes:

```
OperacionFactoring  
EventoFactoring  
FlujoFactoring  
AlertaCobranza
```

Nuevo:

```
ReclamoRecursoFactoring
```

3. Condición con recurso

Un reclamo ordinario solo procede cuando:

```
OperacionFactoring.con_recurso = True
```

Cuando el convenio indica `sin recurso`, el sistema debe:

- Bloquear el registro ordinario.
- Permitir registrar la notificación como controversia.
- Exigir revisión administrativa o legal.
- No asumir automáticamente que Extintores Real debe pagar.

4. Flujo del reclamo

```
Factor notifica deuda
→ reclamo NOTIFICADO
→ revisión interna
  └─ OBJETADO
  └─ ACEPTADO
      → flujo COBRO_RECURSO
      → pago al factor
      → conciliación bancaria
      → reclamo PAGADO
```

5. Monto reclamado

El monto reclamado puede incluir:

- Capital pendiente.
- Intereses.
- Gastos.
- Comisiones.
- Penalidades.

Estos componentes deben separarse cuando el documento del factor los informe.

Se recomienda agregar:

```
capital_reclamado = models.DecimalField(
    max_digits=14,
    decimal_places=2,
    default=Decimal("0.00"),
)

intereses_reclamados = models.DecimalField(
    max_digits=14,
    decimal_places=2,
    default=Decimal("0.00"),
)

gastos_reclamados = models.DecimalField(
    max_digits=14,
    decimal_places=2,
    default=Decimal("0.00"),
)
```

Y validar:

```
monto reclamado
=
capital + intereses + gastos
```

cuando exista desglose.

6. Respuesta al reclamo

Aceptar

Debe registrar:

- Monto aceptado.
- Usuario.
- Fecha.
- Justificación.
- Aprobación necesaria.

Objetar

Debe registrar:

- Motivo.
- Monto objetado.
- Documentos.
- Respuesta enviada.
- Próximo seguimiento.

Aceptar parcialmente

Debe permitir:

```
Monto reclamado:      500.000 CLP
Monto aceptado:       450.000 CLP
Monto objetado:       50.000 CLP
```

7. Flujo financiero

Cuando se acepta el reclamo:

```
FlujoFactoring
tipo = COBRO_RECURSO
dirección = EGRESO
naturaleza = MOVIMIENTO_BANCARIO
```

```
monto = monto aceptado  
estado = ESPERADO
```

Después:

```
MovimientoBancario EGRESO  
→ AplicacionMovimientoBancario  
→ FlujoFactoring COBRO_RECUSO
```

8. Efecto sobre la factura

El pago de Extintores Real al factor:

- No significa que el cliente pagó.
- No debe marcar la factura como pagada por el cliente.
- No debe crear `PagoCliente`.
- No debe reducir artificialmente la deuda del cliente.

En cambio, crea o conserva una obligación de cobro contra el cliente.

El tratamiento detallado de esa deuda continuará en X008-X013.

9. Estado de la operación

Al registrar la invocación del recurso:

```
OperacionFactoring.estado  
→ COBRO_RECUSO
```

Se crea:

```
EventoFactoring.COBRO_CON_RECUSO
```

El modelo actual ya contempla ese tipo de evento y estado.

10. Alertas

El modelo actual incluye:

```
FACTOR_NOTIFICA_DEUDA
```

como tipo de alerta.

La alerta deberá relacionarse con:

- Operación.
- Reclamo.
- Fecha límite.
- Monto.
- Responsable interno.

Se recomienda agregar:

```
RECLAMO_RECURSO_PENDIENTE_RESPUESTA
RECLAMO_RECURSO_PENDIENTE_PAGO
RECLAMO_RECURSO_OBJETADO
```

11. Documentos

Adjuntos posibles:

- Carta del factor.
- Liquidación de deuda.
- Contrato.
- Evidencia de mora.
- Respuesta de Extintores Real.
- Comprobante de pago.
- Comunicaciones con el cliente.

Todos deben utilizar descarga protegida y conservar hash.

12. Servicio

```
services/factoring/recurso.py
```

```
@transaction.atomic
def registrar_reclamo_recurso(
    *,
    operacion_id,
    fecha_notificacion,
    monto_reclamado,
    referencia,
    motivo,
    usuario,
):
    operacion = (
```

```

        OperacionFactoring.objects
        .select_for_update()
        .get(pk=operacion_id)
    )

    # Validar recurso.
    # Validar monto.
    # Crear reclamo.
    # Cambiar estado.
    # Crear evento.
    # Crear alerta.
    # Registrar auditoría.

```

Funciones:

```

registrar_reclamo_recurso()
iniciar_revision_reclamo()
aceptar_reclamo()
objetar_reclamo()
aceptar_parcialmente()
generar_flujo_cobro_recurso()
registrar_pago_recurso()
conciliar_pago_recurso()
anular_reclamo()

```

13. Reglas

1. La operación debe estar activa.
2. El reclamo ordinario requiere `con_recurso=True`.
3. El monto debe ser positivo.
4. El monto reclamado debe tener fundamento.
5. La notificación debe tener fecha.
6. La referencia debe registrarse cuando exista.
7. La respuesta debe respetar permisos.
8. El aceptante no debe ser necesariamente quien registró el reclamo.
9. El pago no supera el monto aceptado.
10. El flujo debe ser de egreso.
11. El movimiento bancario debe ser de egreso.
12. El cliente no se marca como pagador.
13. El pago de recurso no crea `PagoCliente`.
14. Las objeciones conservan documentos.
15. Las anulaciones requieren motivo.
16. Toda transición genera evento.

14. URLs

```
/administracion/factoring/operaciones/<pk>/reclamos/  
/administracion/factoring/operaciones/<pk>/reclamos/nuevo/  
/administracion/factoring/reclamos/<pk>/  
/administracion/factoring/reclamos/<pk>/revisar/  
/administracion/factoring/reclamos/<pk>/aceptar/  
/administracion/factoring/reclamos/<pk>/objetar/  
/administracion/factoring/reclamos/<pk>/pagar/  
/administracion/factoring/reclamos/<pk>/anular/
```

15. Vistas

```
ReclamoRecursoListView  
ReclamoRecursoCreateView  
ReclamoRecursoDetailView  
ReclamoRecursoRevisarView  
ReclamoRecursoAceptarView  
ReclamoRecursoObjetarView  
ReclamoRecursoPagarView  
ReclamoRecursoAnularView
```

16. Permisos

```
view_reclamorecursofactoring  
add_reclamorecursofactoring  
revisar_reclamorecursofactoring  
aceptar_reclamorecursofactoring  
objetar_reclamorecursofactoring  
pagar_reclamorecursofactoring  
anular_reclamorecursofactoring
```

17. Auditoría

Registrar:

- Notificación.
- Operación.
- Condición con recurso.
- Monto reclamado.
- Monto aceptado.
- Monto objetado.

- Usuario.
 - Respuesta.
 - Documentos.
 - Flujo.
 - Movimiento bancario.
 - Estado anterior y nuevo.
-

18. Pruebas

- Reclamo con recurso.
 - Reclamo sin recurso.
 - Monto cero.
 - Desglose correcto.
 - Desglose inconsistente.
 - Aceptación total.
 - Aceptación parcial.
 - Objeción.
 - Pago.
 - Exceso de pago.
 - Movimiento de ingreso incorrecto.
 - Movimiento de egreso válido.
 - Cliente sigue adeudando.
 - Alerta.
 - Anulación.
 - Concurrencia.
-

19. Criterios de aceptación

- El reclamo queda estructurado.
 - Se valida la condición de recurso.
 - Se registra el monto y fundamento.
 - Se puede aceptar u objetar.
 - La aceptación genera un flujo de egreso.
 - El pago se concilia con banco.
 - No se registra como pago del cliente.
 - El historial se conserva.
 - Las alertas se actualizan.
 - La operación queda auditada.
-

8. Estructura de archivos del bloque

```
administracion/  
├── models/  
│   └── factoring.py  
└──
```

```
├─ forms/
│   ├── empresas_factoring.py
│   ├── contactos_factoring.py
│   ├── convenios_factoring.py
│   ├── operaciones_factoring.py
│   ├── flujos_factoring.py
│   ├── pagos_factor.py
│   └── reclamos_recurso.py
│
├─ views/
│   ├── empresas_factoring.py
│   ├── contactos_factoring.py
│   ├── convenios_factoring.py
│   ├── operaciones_factoring.py
│   ├── eventos_factoring.py
│   ├── flujos_factoring.py
│   ├── pagos_factor.py
│   ├── alertas_factoring.py
│   └── reclamos_recurso.py
│
├─ services/factoring/
│   ├── empresas.py
│   ├── convenios.py
│   ├── operaciones.py
│   ├── workflow.py
│   ├── flujos.py
│   ├── pagos_cliente.py
│   ├── alertas.py
│   └── recurso.py
│
├─ selectors/
│   ├── empresas_factoring.py
│   ├── operaciones_factoring.py
│   ├── flujos_factoring.py
│   ├── eventos_factoring.py
│   ├── alertas_factoring.py
│   └── reclamos_recurso.py
│
├─ management/commands/
│   └── generar_alertas_factoring.py
│
├─ templates/administracion/factoring/
│   ├── empresas/
│   ├── contactos/
│   ├── convenios/
│   ├── operaciones/
│   ├── eventos/
│   ├── flujos/
│   ├── pagos_factor/
│   └── alertas/
```



```
|   └─ reclamos/
|
|   └─ static/administracion/js/
|       ├── operacion_factoring.js
|       └─ flujos_factoring.js
|
|   └─ tests/factoring/
|       ├── test_empresas.py
|       ├── test_convenios.py
|       ├── test_operaciones.py
|       ├── test_workflow.py
|       ├── test_flujos.py
|       ├── test_pagos_factor.py
|       ├── test_alertas.py
|       └─ test_recurso.py
```

9. Migraciones previstas

Migración 1 — Empresas y contactos

Crear:

```
ContactoEmpresaFactoring
ConvenioFactoring
```

Agregar a `EmpresaFactoring`:

```
nombre de fantasía
dirección
sitio web
código interno
```

Migración 2 — Operación

Agregar:

```
convenio
referencia de operación
moneda
notificación al cliente
porcentajes pactados
```

snapshot contractual
fechas estimadas

Migración 3 — Eventos

Agregar:

estado anterior
estado nuevo
monto
referencia
datos JSON
correlation_id
nuevos tipos de evento

Migración 4 — Flujos

Agregar:

naturaleza
requiere conciliación bancaria
estado
fecha esperada
fecha de ejecución
campos de anulación

La migración de datos debe clasificar los flujos existentes según su tipo.

Migración 5 — Alertas

Agregar a `AlertaCobranza`:

nuevos tipos
clave de control
fecha límite
asignación
motivo de resolución

Migración 6 — Reclamo con recurso

Crear:

```
ReclamoRecursoFactoring
```

10. Índices recomendados

EmpresaFactoring

```
models.Index(  
    fields=["activa", "nombre"],  
)
```

ContactoEmpresaFactoring

```
models.Index(  
    fields=["empresa", "tipo", "activo"],  
)
```

ConvenioFactoring

```
models.Index(  
    fields=["empresa", "activo"],  
)  
  
models.Index(  
    fields=["fecha_inicio", "fecha_fin"],  
)
```

OperacionFactoring

```
models.Index(  
    fields=["empresa_factoring", "estado"],  
)  
  
models.Index(  
    fields=["estado", "fecha_vencimiento"],  
)  
  
models.Index(  

```

```
        fields=["con_recurso", "estado"],
    )
```

EventoFactoring

```
models.Index(
    fields=["operacion", "-fecha"],
)

models.Index(
    fields=["tipo", "-fecha"],
)

models.Index(
    fields=["correlation_id"],
)
```

FlujoFactoring

```
models.Index(
    fields=["operacion", "tipo", "estado"],
)

models.Index(
    fields=["fecha_esperada", "estado"],
)

models.Index(
    fields=["requiere_conciliacion_bancaria", "estado"],
)
```

ReclamoRecursoFactoring

```
models.Index(
    fields=["operacion", "estado"],
)

models.Index(
    fields=["fecha_limite_respuesta", "estado"],
)
```

11. Definición de terminado del bloque

X001–X007 se considerará terminado cuando:

- Se pueden administrar empresas de factoring.
- Se pueden administrar contactos.
- Existen convenios versionados.
- Las condiciones se congelan por operación.
- Se puede ceder una factura elegible.
- No se cede más que el saldo disponible.
- La factura cambia a cedida.
- Las transiciones están controladas.
- Cada transición genera evento.
- Anticipo, comisión, retención y liquidación están separados.
- Los descuentos no generan movimientos bancarios ficticios.
- Los flujos bancarios pueden conciliarse.
- Se puede confirmar el pago del cliente al factor.
- Ese pago no crea un ingreso bancario propio.
- Se soportan pagos parciales al factor.
- Se generan alertas de mora.
- Las alertas no se duplican.
- Se puede registrar un reclamo con recurso.
- El reclamo puede aceptarse u objetarse.
- El pago con recurso se registra como egreso.
- El pago con recurso no se considera pago del cliente.
- Las operaciones son transaccionales.
- Las acciones quedan auditadas.
- Las pruebas automáticas pasan.

12. Riesgos del bloque

Riesgo	Impacto	Tratamiento
Mezclar cesión, cobranza y banco	Alto	Modelos y estados separados
Modificar condiciones históricas	Alto	Snapshot y versionado
Ceder una factura pagada	Alto	Validar saldo cedible
Ceder más que el saldo	Alto	Control transaccional
Modelar comisión como egreso ficticio	Alto	Naturaleza no bancaria
Modelar retención como ingreso	Alto	Flujo de retención separado
Confundir pago al factor con ingreso propio	Alto	Estado “no requiere conciliación”
Cerrar operación sin liquidación	Alto	Validación de flujos
Cambiar estado manualmente	Alto	Servicio de workflow

Riesgo	Impacto	Tratamiento
Perder eventos anteriores	Alto	Eventos inmutables
Duplicar alertas	Medio/alto	Clave de control
Activar mora antes del plazo	Alto	Vencimiento y días de gracia
Reclamo sobre operación sin recurso	Alto	Bloqueo y revisión especial
Pagar recurso y marcar cliente pagado	Alto	Flujo de egreso separado
Sobreconciliar un flujo	Alto	Control de ambos saldos
Convenios con fórmulas diferentes	Alto	Configuración contractual
One-to-one impide cesiones parciales futuras	Medio/alto	Limitación documentada
Cambios concurrentes de estado	Alto	<code>select_for_update()</code>

13. Resultado consolidado

Al finalizar este bloque existirá el ciclo:

```

Empresa de factoring
→ convenio
→ factura cedida
→ operación contractual
→ eventos
→ anticipo y componentes financieros
→ pago del cliente al factor
├─ liquidación normal
├─ mora
│   → notificación
│   → cobro con recurso
→ conciliación bancaria de flujos propios

```

La salida habilita la **Parte 12: X008-X013**, correspondiente a:

```

X008 – Detectar pago directo de factura cedida
X009 – Devolución al factoring
X010 – Seguimiento de devolución
X011 – Dashboard y reportes de factoring
X012 – Pruebas integrales de factoring
X013 – Documentación y cierre del módulo

```